

CS & IT ENGINEERING



Operating System

Deadlock

Lecture -1

By- Vishvadeep Gothi sir



Recap of Previous Lecture



Topic

Reader-Writer Problem

Topic

Dining Philosopher Problem

Topics to be Covered



Topic

Deadlock

Topic

Deadlock Prevention

Topic

Deadlock Avoidance



Topic : Operations on Resources

3 Operations on resources:

1. Request
2. Use
3. Release
4. *wait*



Topic : Deadlock



If two or more processes are waiting for such an event which is never going to occur

	Hold	wait
P ₁	Printer	HDD
P ₂	HDD	Camera
P ₃	Camera	Printer



Topic : Deadlock





Topic : ~~Writer()~~ & ~~Reader()~~ Processes

Producer()



wait(mutex)
wait(Empty) ←

//
signal(mutex)
signal(Full)

Consumer()



wait(Full)
→ wait(mutex)

//
signal(mutex)
signal(Empty)

There is a possibility of deadlock when buffer full.

mutex = 1
full = n-1
empty = 0



Topic : ~~Writer()~~ & ~~Reader()~~ Processes

Producer()



→ wait(Empty)
→ wait(mutex)

//
signal(mutex)
signal(Full)

Consumer()

→ wait(mutex)
wait(Full)

//
signal(mutex)
signal(Empty)

There is a possibility of deadlock when buffer is empty

mutex = 10

full = 0

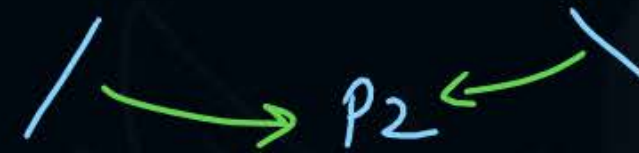
Empty = ~~n~~ n-1



Topic : Dining Philosopher Problem Solution

$\text{wait}(\text{ch}[i])$
 $\rightarrow \text{wait}(\text{ch}[(i+1)\%k])$

if all philosophers
pick one chopstick
then there will be deadlock.





Topic : Dining Philosopher Problem Solution

Some of the ways to avoid deadlock are as follows –

1. There should be at most $(k-1)$ philosophers on the table



Topic : Dining Philosopher Problem Solution

Some of the ways to avoid deadlock are as follows –

1. There should be at most $(k-1)$ philosophers on the table
2. A philosopher should only be allowed to pick their chopstick if both are available at the same time





Topic : Dining Philosopher Problem Solution

Some of the ways to avoid deadlock are as follows –

1. There should be at most $(k-1)$ philosophers on the table
2. A philosopher should only be allowed to pick their chopstick if both are available at the same time
3. One philosopher should pick the left chopstick first and then right chopstick next; while all others will pick the right one first then left one



Topic : Deadlock



If two or more processes are waiting for such an event which is never going to occur

	Hold	wait
P1	Printer	HDD
P2	HDD	Camera
P3	Camera	Printer



Topic : Necessary Conditions for Deadlock

Deadlock can occur only when all following conditions are satisfied:

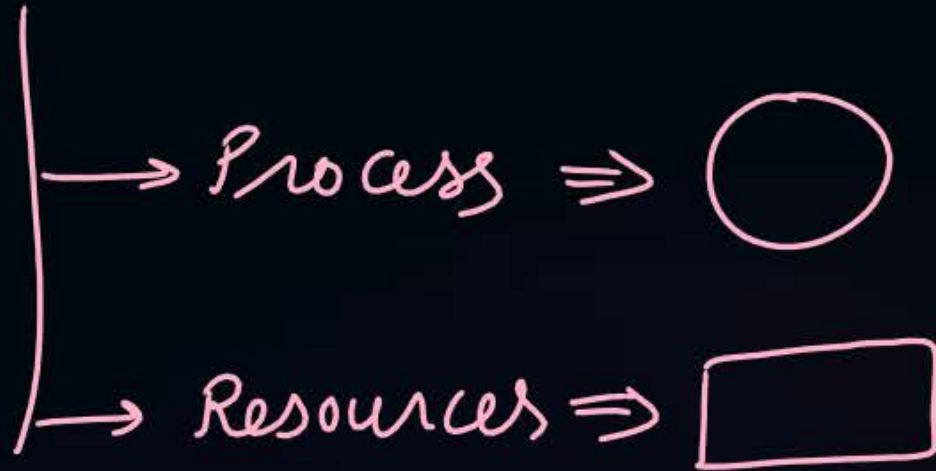
1. Mutual Exclusion → a resource can be used only by one process at a time.
2. Hold & Wait → all deadlocked processes should hold at least one resource and should wait for at least one resource.
3. No-preemption → The forcible preemption of resource is not allowed.
4. Circular Wait → all deadlocked processes should wait for each other in circular manner.



Topic : Resource Allocation Graph

→ it is a directed graph which denotes allocation and wait of resources for processes.

vertices (nodes)



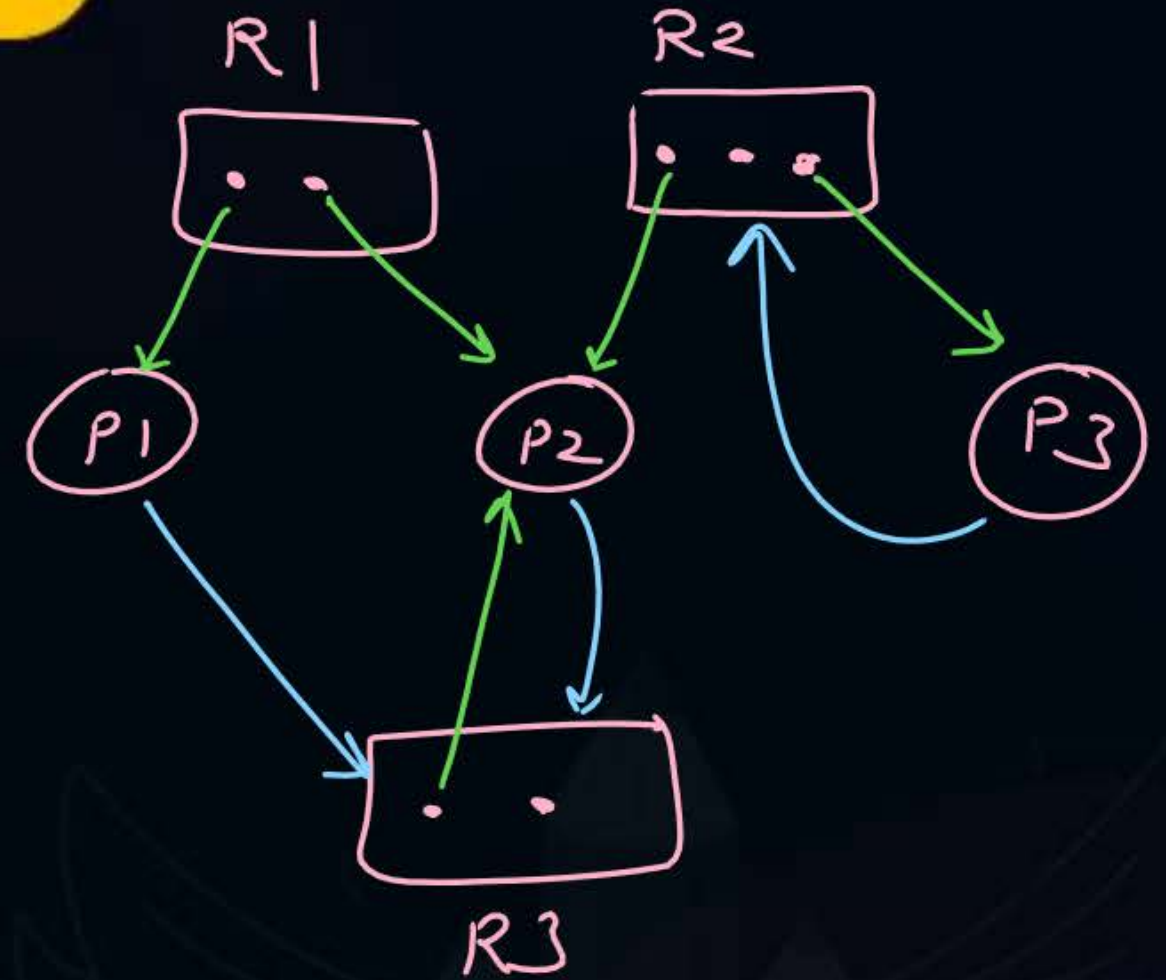
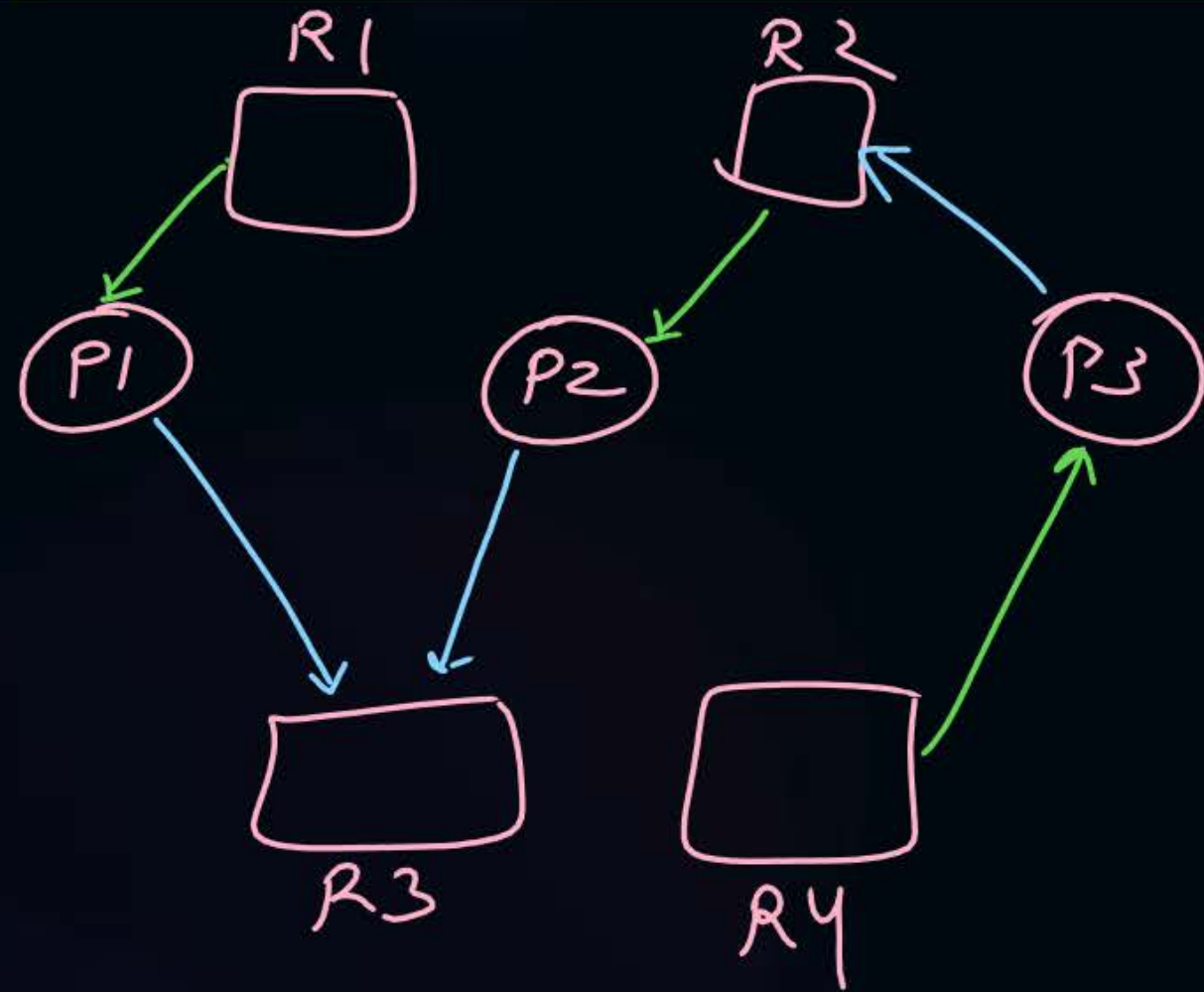

3 instances
of a resource

Edges :-

- allocation ⇒ from resource to process
(from instance to process)
- Request ⇒ from process to resource



Topic : Resource Allocation Graph





Topic : Recovery From Deadlock

1. Make Sure that deadlock never occur
 - Prevent the system from deadlock or avoid deadlock
2. Allow deadlock, detect and recover
3. Pretend that there is no any deadlock



Topic : Deadlock Prevention

OS makes sure one of the four necessary conditions never satisfies.

1. Mutual Exclusion:-

either make all processes independent or have enough no. of resources in system such that each process can have its own resources.

⇒ Practically not possible

2. Hold & wait:-

A process should either hold all required resources or should wait for all resources.

⇒ Processes may starve

3. No preemption:-

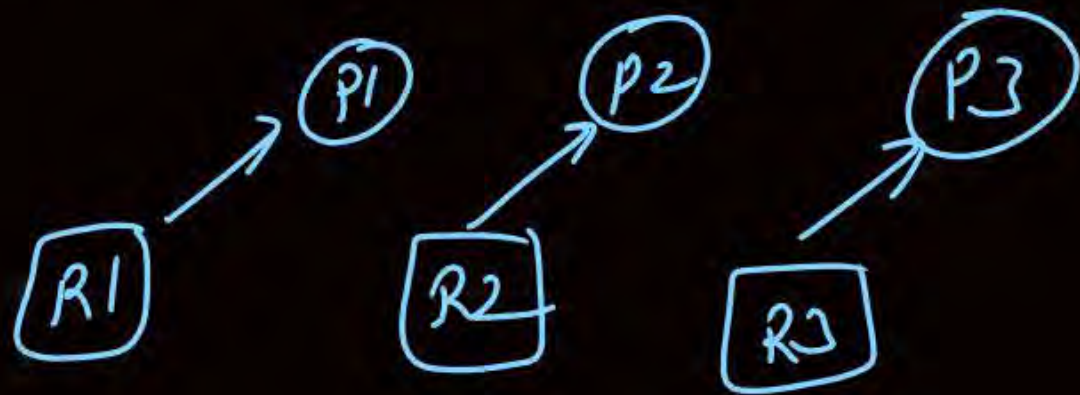
Resources can be preempted from processes.

- Victim process may become unstable
- Processes may starve

4. Circular wait:-

All the resources are given numbers and each process can request for a resource R_j while holding R_i only when $i < j$.

if $i > j$; then first process releases R_i then tries to acquire R_j first then R_i next.





2 mins Summary

Topic

Deadlock

Topic

Deadlock Prevention

Topic

Deadlock Avoidance



Happy Learning

THANK - YOU

